K. R. Mangalam University (KRMU)

School of Engineering & Technology

CAPSTONE PROJECT – PHASE I REPORT

TaskFlow

A Concurrent Full-Stack Task & Project Management System

Your day, in order.

Course	ENSP465 — New Age Programming Languages Lab
Project Title	TaskFlow – A Concurrent Full-Stack Task & Project Management System
Student Name	Sumukhi Pandey
Roll Number	2301010184
Program / Year	B.Tech, 4th Year
Semester / Section	VII / C
Academic Year	2026-27
Faculty Name	Dr. Hemraj Kumawat
School	School of Engineering & Technology, KRMU
Submission Date	22 October 2026

Primary language: Go | Database: SQLite | Frontend: React

1. Problem Statement & Motivation

1.1 The Problem

Students, freelancers, early-career professionals and small teams juggle many parallel commitments. A final-year student may handle coursework, a capstone project, internship applications and examinations at once; a freelancer may track several clients, each with separate deliverables and deadlines. Without a structured system, deadlines are missed not for lack of effort but for lack of visibility: it is hard to see what is due soon, what is in progress and what is finished.

1.2 Limitations of Existing Tools

Trello, Todoist, Notion, Microsoft To Do and spreadsheet boards solve parts of this problem, but they fall short for lightweight, local-first, privacy-focused use:

- **Cloud dependency.** Task data is stored on third-party servers the user cannot inspect, and most tools degrade heavily without internet access.
- **Subscription gating.** Advanced filtering, analytics, calendar views and activity history are often placed behind paid tiers, a barrier for students and independent workers.
- **Weight and complexity.** General-purpose workspaces are slow to configure when a user only needs tasks, projects, filters and statistics.
- **Weak structure in spreadsheets.** They offer no access control, consistent validation, automatic overdue detection or activity history.
- **Opaque internals.** Closed products cannot serve as a learning platform for how a request travels from the screen to a database and back.

1.3 Real-World Context and Motivation

TaskFlow targets personal productivity and small-team task orchestration without cloud dependency or subscription cost. It runs locally as a Go service with an embedded SQLite database, so data stays on the user's machine. A task backend is also naturally concurrent: several browser tabs, dashboard refreshes and analytics queries can arrive together, making it a good vehicle for the goroutine, channel and context patterns taught in ENSP465. The aim is a **simple, fast, secure and concurrent backend** that can later grow to support real-time features.

2. Proposed Solution & Language Justification

2.1 Project Overview

TaskFlow is a local full-stack web application. Authenticated users create, update, tag, filter, search and sort tasks (statuses To Do, In Progress and Completed) within projects, and see dashboard statistics, a month calendar, analytics charts and an activity timeline. A Go REST API handles business logic, authentication and persistence in SQLite; a React single-page application provides the interface, with a light/dark theme and clear loading, error and empty states.

2.2 Language Chosen

Primary language: Go. All server-side logic (HTTP handling, authentication, validation, persistence, concurrency and analytics) is written in Go. **Secondary: React with Vite**, used only for the user interface, so the substantive engineering stays in Go as the course requires.

2.3 Language Justification

Go strength	Why it suits TaskFlow
1. Goroutines and channels	`net/http` serves every connection in its own goroutine, so overlapping requests (list queries, status updates, dashboard and calendar fetches) run concurrently without a thread-pool framework. Goroutines start with a few kilobytes of stack, so thousands can coexist cheaply. The dashboard can compute independent statistics (counts by status, overdue count, priority split) in separate goroutines and merge results via channels or `sync.WaitGroup`. The same primitives make future real-time feeds a natural extension.
2. Standard library	`net/http`, `database/sql`, `encoding/json` and `crypto/*` (password hashing, `crypto/rand` tokens, constant-time comparison) are enough for a secure REST API with authentication. Fewer dependencies mean a smaller attack surface and an easily audited, privacy-focused codebase.
3. Static typing, explicit errors	Compile-time checks catch mismatches between request payloads, models and database rows. The `if err != nil` convention forces every failure path (validation, constraint violations, missing rows, expired sessions) to be handled and mapped to an HTTP status, reducing silent data loss.
4. Fast builds, single binary	Quick compilation supports a tight edit-build-test loop. The backend becomes one static executable that cross-compiles to Windows, Linux and macOS, matching the local-first goal: run one binary and open the browser. Low memory use and an efficient scheduler suit I/O-bound, database-backed APIs.
5. Built-in tooling	`go test`, `go vet`, `gofmt`, `httptest` and the race detector (`go test -race`) ship with the toolchain, supporting the planned unit tests and verification of concurrent code.

2.4 Key Libraries and Technology Stack

Layer	Technology	Role
Backend	Go, net/http (ServeMux), database/sql	REST API, routing, middleware chain, SQL access
Database	SQLite (taskflow.db) via modernc.org/sqlite (pure Go, no C compiler) or mattn/go-sqlite3	Persistent storage of users, projects, tasks and activity
Security	PBKDF2-HMAC-SHA256 (golang.org/x/crypto/pbkdf2); session cookie	Password hashing and session-based access control
Router (optional)	go-chi/chi or gorilla/mux	Path parameters; core works with ServeMux alone
Frontend	React + Vite (dev server on port 5173)	Interactive pages consuming the API with credentials
Testing	testing, net/http/httptest, in-memory SQLite	Unit and handler tests

3. Objectives & Scope

3.1 Objectives

1. **Concurrent REST API.** A Go API that serves at least **50 simultaneous authenticated requests** with correct session handling, verified by a load test and the race detector (zero reported races).
2. **Secure authentication.** Email-and-password login/logout with PBKDF2-SHA256 hashing, per-user salts and session cookies; middleware must reject **100%** of unauthenticated requests to protected endpoints (verified by tests).
3. **Complete data management.** Full CRUD for tasks and projects with server-side validation, tags, status/priority filters, search and sorting, all **scoped to the authenticated user**.
4. **Insights.** Dashboard, month calendar, analytics and activity timeline, responding in under **200 ms** for about 1,000 tasks on a typical laptop.
5. **Quality.** At least **70% unit-test coverage** on core Go packages and **additive** SQLite migrations that upgrade an existing database without data loss.

3.2 In Scope (Phases II and III)

Login/logout with persistent sessions; task and project CRUD; filtering, search and sorting; overdue detection; dashboard, calendar and analytics; activity log; responsive React UI with theme, validation and loading/error/empty states; Go unit and handler tests; additive SQLite migrations and seeding.

3.3 Out of Scope

User registration, password reset and email verification; multi-user real-time collaboration (WebSockets); mobile apps; production cloud deployment; third-party OAuth; payment features; AI suggestions.

4. System Architecture & Design

4.1 High-Level Architecture

TaskFlow is a three-tier client-server system. The **presentation tier** is a React SPA; the **application tier** is a Go REST API with a middleware chain (logging, panic recovery, authentication) in front of the modules; the **data tier** is an embedded SQLite database accessed through `database/sql`.

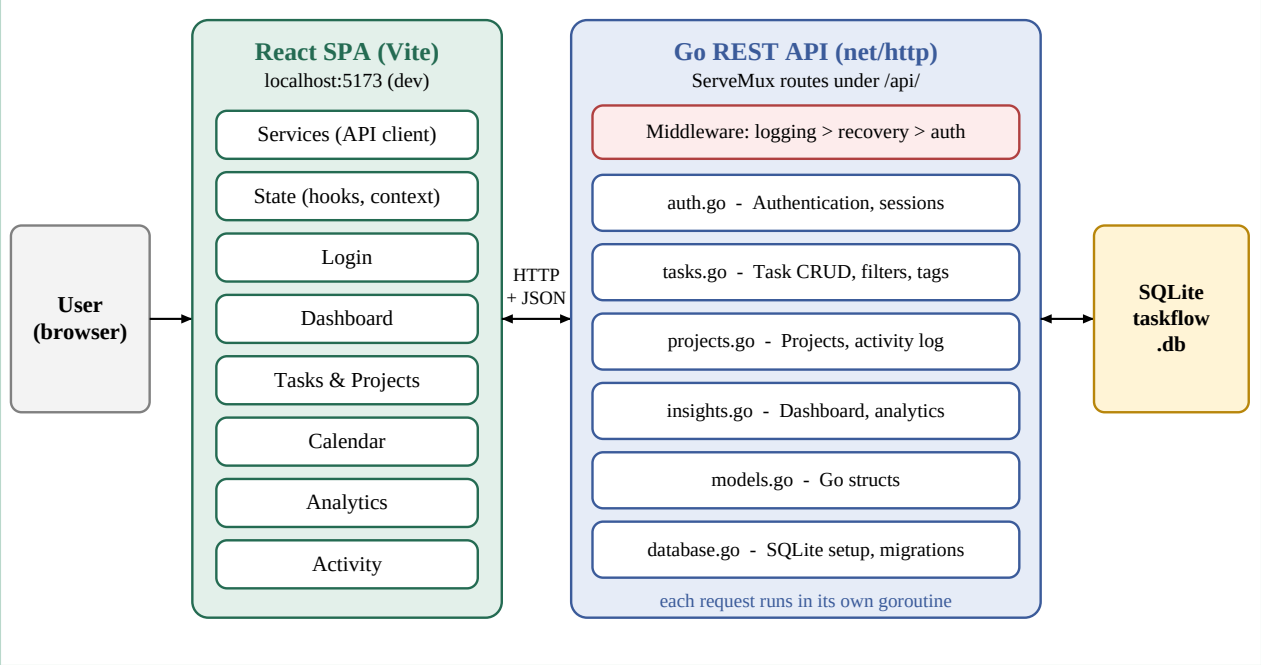

Figure 1 – High-level architecture of TaskFlow

4.2 Component Description

Module	Responsibilities
Auth (auth.go)	PBKDF2-HMAC-SHA256 hashing with a random per-user salt from `crypto/rand` and constant-time verification. On login a random session token is stored server-side and sent in an HttpOnly, SameSite cookie, so sessions survive restarts. Middleware validates the cookie and puts the user ID in the request `context.Context`. Endpoints: POST /api/auth/login, POST /api/auth/logout, GET /api/auth/me. No self-registration; accounts are seeded.
Tasks (tasks.go)	CRUD on /api/tasks. Every query carries `WHERE user_id = ?`. Validation covers title, status (To Do, In Progress, Completed), priority, due date and tags, returning HTTP 400 with JSON errors. Filters, search and sorting become **parameterised** SQL with allow-listed sort columns. Overdue is computed (due date passed and not Completed), never stored.
Projects (projects.go)	CRUD on /api/projects, per-project task counts and completion progress, and a documented policy for tasks when a project is deleted. Writes activity events.
Insights (insights.go)	Dashboard totals, per-status and priority counts, overdue count; calendar tasks grouped by due date for a month; analytics series; activity timeline. Independent queries run in parallel goroutines under a `context` deadline and are merged before responding.
Database (database.go, models.go)	Go structs plus `initializeDatabase`: tables users, projects, tasks and activity_events with foreign keys (enabled via PRAGMA), indexes on user_id, status and due_date, WAL mode and a busy timeout. **Additive** migrations only (CREATE TABLE IF NOT EXISTS, ALTER TABLE ADD COLUMN) plus seeding on an empty database.
React frontend	**Services:** fetch wrapper with `credentials: include` and central error handling. **State:** hooks and context for auth, theme and cached lists with loading/error flags. **Screens:** Login, Dashboard, Tasks, Projects, Calendar, Analytics, Activity, with client-side validation and a persisted light/dark theme.

4.3 Data Flow

Each user action enters one of five processes; all share the single SQLite store, and create, update and delete operations also feed the activity log.

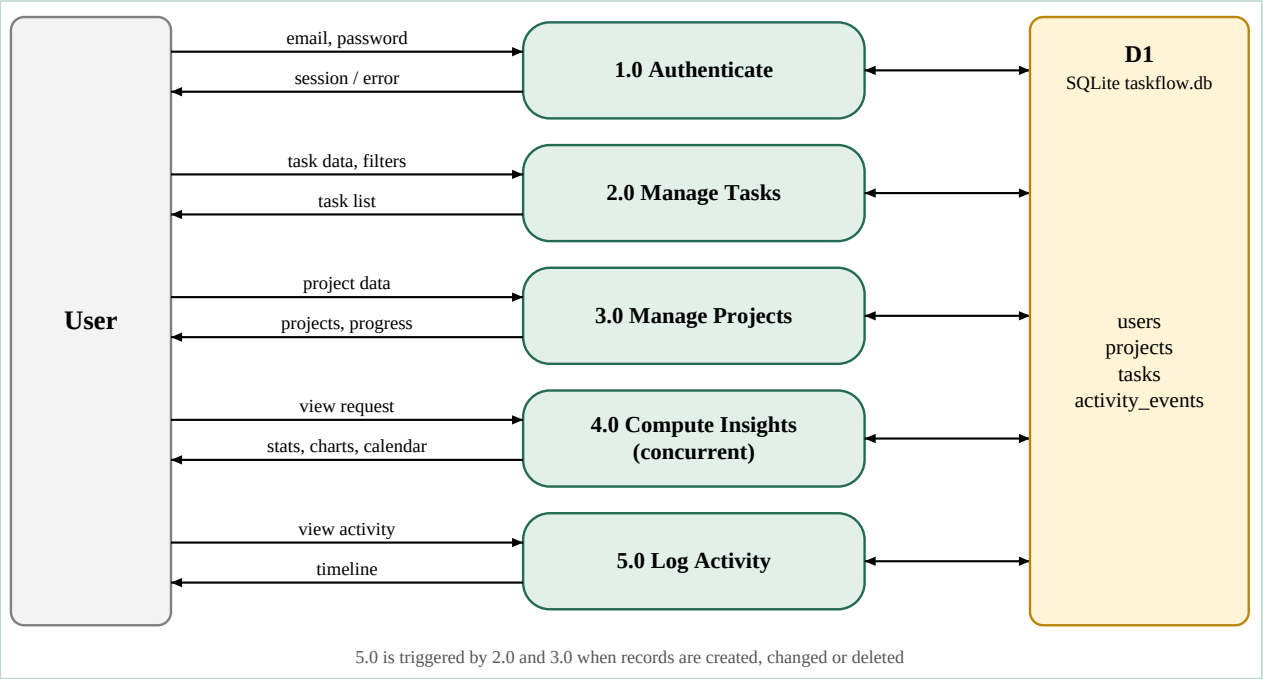

Figure 2 – Data Flow Diagram, Level 1 (Level 0: User <-> TaskFlow System <-> SQLite)

The main workflow (Login, Fetch Dashboard, Create/Update Task, Analytics refresh) is shown below.

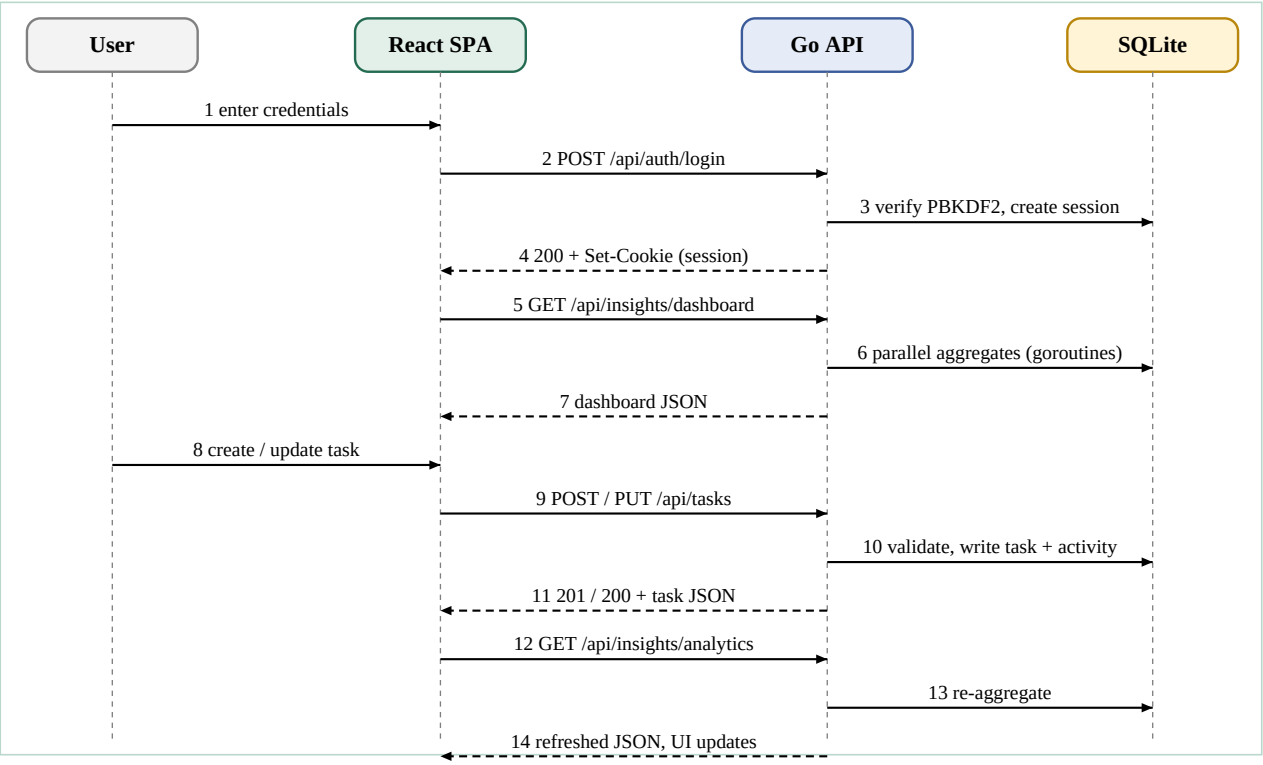

Figure 3 – Sequence diagram of the main workflow

4.4 Database Design

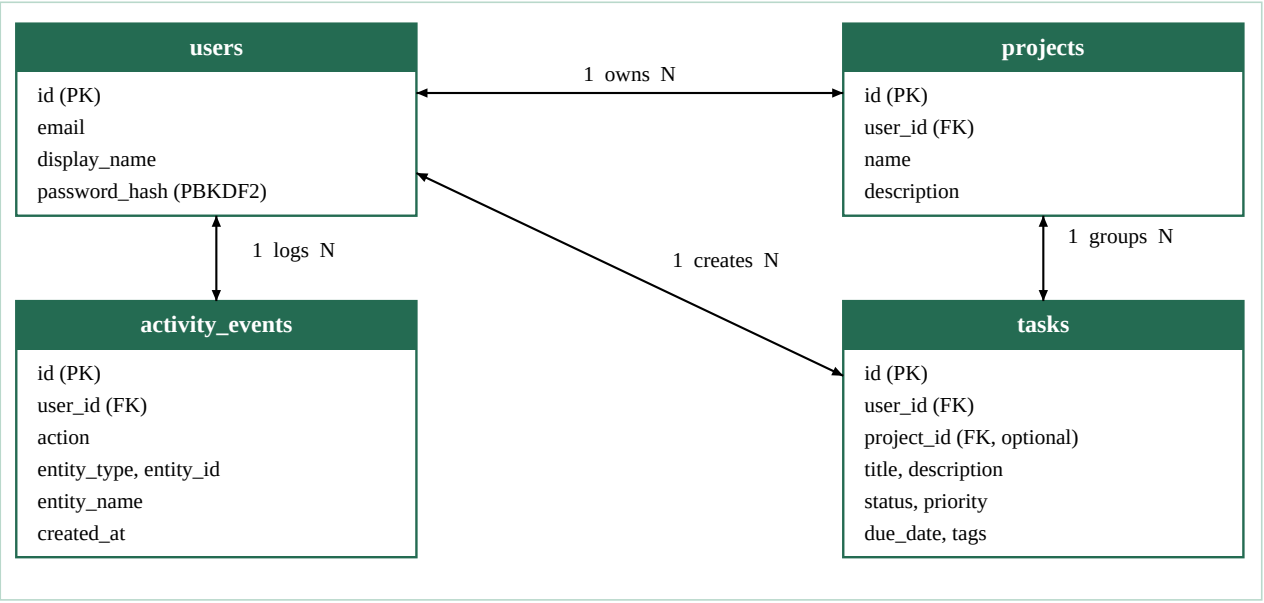

Figure 4 – Entity-relationship diagram (one user owns many projects, tasks and activity events)

4.5 Input-Process-Output (IPO)

Feature	Input	Process	Output
Login / logout	Email, password / session cookie	Look up user, PBKDF2 verify, create or delete session	Session cookie, profile / cleared cookie
Create / update task	Title, description, due date, status, priority, project, tags	Authenticate, validate, INSERT or UPDATE, record activity	Task JSON (201 / 200)
List tasks	Search text, filters, sort	Build parameterised query scoped to user	Filtered, sorted task list
Delete task	Task ID	Check ownership, DELETE, record activity	204 No Content
Dashboard, analytics	Authenticated request	Concurrent aggregate queries	Statistics and chart-ready JSON
Calendar, activity	Month (YYYY-MM) / pagination	Group by due date / read log for user	Date-indexed tasks / timeline

4.6 Key Data Structures and Go-Specific Design

```
type Task struct {
    ID        int64      `json:"id"`
    UserID    int64      `json:"-"`
    ProjectID *int64     `json:"projectId,omitempty"`
    Title     string     `json:"title"`
    Status    string     `json:"status"`     // To Do | In Progress | Completed
    Priority  string     `json:"priority"`
    DueDate   *time.Time `json:"dueDate,omitempty"`
    Tags      []string   `json:"tags"`
    Overdue   bool       `json:"overdue"`    // computed, not stored
}
```

- **Goroutines and channels:** one goroutine per request via net/http; dashboard aggregates fan out to goroutines and join through channels or `sync.WaitGroup`, cutting latency versus sequential queries.
- **Context:** `r.Context()` is passed to `QueryContext` and `ExecContext`, so a client disconnect or timeout cancels in-flight queries; it also carries the authenticated user ID.
- **Prepared statements and middleware:** placeholders prevent SQL injection; cross-cutting concerns compose as `func(http.Handler) http.Handler`: Logging, Recovery, Auth, Router.
- **Interfaces:** small interfaces such as `activityWriter` (needing only an `Exec` method) and repository interfaces decouple handlers from SQLite and enable test doubles.
- **Safety and errors:** shared state is minimal (`sync.RWMutex` where needed, verified by `go test -race`); wrapped errors (`errors.Is/As`) map to HTTP statuses in one place.

5. Work Plan & Timeline

This is an **individual project**: all modules (backend, database, frontend, testing and documentation) are handled by **Sumukhi Pandey**. Dates after 04 October 2026 are tentative and will be aligned with the faculty schedule.

#	Milestone	Phase	Tentative dates	Deliverable
M1	Proposal and design	I	01 - 04 Oct 2026	This document: problem, scope, architecture, diagrams
M2	Core backend, auth, database	II	20 Oct - 02 Nov 2026	Router, middleware, migrations and seeding, PBKDF2 login/logout
M3	Tasks, projects, insights	II	03 - 15 Nov 2026	CRUD, validation, filters, tags, dashboard, calendar, analytics, activity
M4	React integration and intermediate demo	II	16 - 25 Nov 2026	End-to-end working demo of all core screens
M5	Testing and concurrency verification	III	26 Nov - 10 Dec 2026	Unit and handler tests, 50+ request load test, race-detector runs
M6	Polish, final report and viva	III	11 - 28 Dec 2026	Responsive UI, documentation, demo script, viva preparation

Phase II (late October - November): working Go API with session authentication, SQLite persistence with additive migrations, task and project CRUD with filters, search, sorting and tags, insights endpoints, and the React UI connected to the API, shown in an intermediate demo.

Phase III (December): complete test suite with coverage and race-detector results, concurrency load-test evidence, a polished responsive UI, the final report and README, and viva preparation.

6. References

[1] The Go Authors, "net/http," *Go Packages*. [Online]. Available: https://pkg.go.dev/net/http. Accessed: 02 October 2026.

[2] The Go Authors, "database/sql," *Go Packages*. [Online]. Available: https://pkg.go.dev/database/sql. Accessed: 02 October 2026.

[3] S. Gerrand, "Go Concurrency Patterns: Context," *The Go Blog*. [Online]. Available: https://go.dev/blog/context. Accessed: 02 October 2026.

[4] S. Gerrand and S. Munroe, "Go Concurrency Patterns: Pipelines and cancellation," *The Go Blog*. [Online]. Available: https://go.dev/blog/pipelines. Accessed: 02 October 2026.

[5] The Go Authors, "Effective Go," *The Go Programming Language*. [Online]. Available: https://go.dev/doc/effective_go. Accessed: 02 October 2026.

[6] modernc.org, "sqlite," *Go Packages*. [Online]. Available: https://pkg.go.dev/modernc.org/sqlite. Accessed: 02 October 2026.

[7] SQLite Consortium, "SQLite Documentation," *sqlite.org*. [Online]. Available: https://www.sqlite.org/docs.html. Accessed: 02 October 2026.

[8] SQLite Consortium, "SQLite Foreign Key Support," *sqlite.org*. [Online]. Available: https://www.sqlite.org/foreignkeys.html. Accessed: 02 October 2026.

[9] Meta Open Source, "React Documentation," *react.dev*. [Online]. Available: https://react.dev/learn. Accessed: 02 October 2026.

[10] Vite Contributors, "Getting Started," *vite.dev*. [Online]. Available: https://vite.dev/guide/. Accessed: 02 October 2026.

[11] K. Moriarty, B. Kaliski, and A. Rusch, "PKCS #5: Password-Based Cryptography Specification Version 2.1," RFC 8018, IETF, Jan. 2017. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8018. Accessed: 02 October 2026.

[12] OWASP Foundation, "Session Management Cheat Sheet," *OWASP Cheat Sheet Series*. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html. Accessed: 02 October 2026.

[13] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan, "Local-first software: You own your data, in spite of the cloud," in *Proc. ACM SIGPLAN Onward!*, 2019, pp. 154-178, doi: 10.1145/3359591.3359737. Accessed: 02 October 2026.

[14] A. A. A. Donovan and B. W. Kernighan, *The Go Programming Language*. Boston, MA, USA: Addison-Wesley, 2015.